

SKILL 7

1. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Sol:- #include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main()

{

pid_t pid;

int status;

printf("Parent Process\n");

printf("Parent PID: %d\n", getpid());

pid = fork();

if (pid < 0)

{

perror("fork failed");

return 1;

}

```
if (pid == 0)
{
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    sleep(2);

    printf("Child process completed\n");
    exit(0);
}
else
{
    printf("\nParent waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited normally\n");
        printf("Exit status: %d\n", WEXITSTATUS(status));
    }

    printf("Parent process completed\n");
}

return 0;
```

```
}
```

OUTPUT:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

/* ----- Built-in Commands ----- */

void builtin_help()
{
    printf("\nAvailable Built-in Commands:\n");
    printf("help   - Show available commands\n");
    printf("pwd    - Print current directory\n");
    printf("clear  - Clear screen\n");
    printf("exit   - Exit shell\n\n");
}
```

```
===== Mini Shell =====
Type 'exit' to quit.
```

```
MiniShell> ls -l
Invalid Built-in Command: ls -l
MiniShell> help

Available Built-in Commands:
help   - Show available commands
pwd    - Print current directory
clear  - Clear screen
exit   - Exit shell
```

```
MiniShell> pwd
/Users/apple/OSSP/skill
MiniShell> clear

MiniShell> █
```